Homework: Predicting Marine Fish Life-History Strategies with a Multilayer Perceptron

Overview

Marine fish communities differ in their life-history strategies. Some communities are dominated by **fast-living species** that tend to grow quickly and mature earlier, while others are dominated by **slow-living species** that tend to mature later and live longer.

In this homework, you will use environmental conditions to predict whether a marine fish community is classified as:

- **slow-life-history community**
- **fast-life-history community**

You will build a **multilayer perceptron (MLP)** using R `torch` and compare it with a logistic-regression baseline.

Learning objectives

By the end of this assignment, you should be able to:

1. Prepare tabular environmental data for a neural network.
2. Explain why feature scaling matters for MLPs.
3. Build and train an MLP for binary classification.
4. Use a validation set rather than the test set to make modeling decisions.
5. Diagnose underfitting and overfitting from training history.
6. Apply early stopping and regularization.
7. Compare an MLP with a simpler linear model.
8. Interpret model performance in an ecological context.

Dataset

The file provided with this assignment is:

`FishTraits_MLP_Student.csv`

It contains the following columns:

Variable	Description
`site_id`	Unique sampling/grid-cell identifier
`longitude`, `latitude`	Site coordinates
`Depth`	Water depth
`SBT`	Mean sea-bottom temperature
`SBS`	Mean sea-bottom salinity
`Chl`	Mean chlorophyll-a concentration
`SBT_var`	Seasonal variability in bottom temperature
`Chl_var`	Seasonal variability in chlorophyll-a
`Fishing`	Fishing-pressure indicator

Variable	Description
`life_history_class`	Response class: `slow` or `fast`
`split`	Instructor-provided data split: `train`, `validation`, or `test`
`spatial_block`	Spatial-block identifier reserved for later spatial-validation exercises

Use only these seven environmental predictors:

```
Depth
SBT
SBS
Chl
SBT_var
Chl_var
Fishing
```

Do **not** use `site_id`, `longitude`, `latitude`, `split`, or `spatial_block` as predictors.

For modeling, encode:

- `slow` = 0
- `fast` = 1

The dataset already contains instructor-defined training, validation, and test sets. Use the `split` column exactly as provided.

Submission

Submit your completed R Markdown document with:

- all code chunks executed,
- all requested figures,
- written answers to every question,
- and your final model comparison.

Your written answers should explain **what the model is doing and what the results mean**, not simply report numbers.

1. Load packages and confirm `torch` works

This assignment uses R `torch`.

If `torch` has not already been installed, run this once in the Console:

```
install.packages("torch")
```

Complete any LibTorch/Lantern installation prompts, restart RStudio, and then run the code below.

```
library(torch)

x_check <- torch_tensor(
  c(1, 2, 3),
  dtype = torch_float32()
)
```

```r
print(x_check)
```

```
## torch_tensor
##  1
##  2
##  3
## [ CPUFloatType{3} ]
```

If this prints a tensor, continue.

2. Load and inspect the data

```r
data_url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/FishTraits.csv"

fish <- read.csv(
  data_url,
  stringsAsFactors = FALSE
)

cat("Rows:", nrow(fish), "\n")
```

```
## Rows: 4012
```

```r
cat("Columns:", ncol(fish), "\n")
```

```
## Columns: 13
```

```r
head(fish)
```

```
##   site_id longitude latitude    Depth      SBT      SBS       Chl    SBT_var  Chl_var    Fishing
## 1     630   170.625   53.375 306.3636 2.413229 34.44804 0.9777634 0.11965475 1.350683 0.12943356
## 2     876   170.875   52.875 186.2000 2.420966 34.44706 0.9467273 0.08637959 1.384809 0.05445741
## 3     877   170.875   53.125 148.1429 2.412065 34.44791 0.8084304 0.09270651 1.166292 0.10551741
## 4     878   170.875   53.375 177.3333 2.483322 34.43353 0.9578087 0.10205390 1.420011 0.12451150
## 5    1621   171.625   53.125 458.0000 2.266274 34.47752 0.6957548 0.09045686 1.286549 0.05673918
## 6    2116   172.125   52.875 331.6364 2.399845 34.44250 0.7173181 0.09068991 1.529178 0.10220994
##   life_history_class split spatial_block
## 1               slow train        116_47
## 2               slow train        116_47
## 3               slow train        116_47
## 4               slow train        116_47
## 5               slow train        117_47
## 6               slow train        117_47
```

```r
str(fish)
```

```
## 'data.frame':    4012 obs. of  13 variables:
##  $ site_id           : int  630 876 877 878 1621 2116 2364 2365 2611 2613 ...
##  $ longitude         : num  171 171 171 171 172 ...
##  $ latitude          : num  53.4 52.9 53.1 53.4 53.1 ...
##  $ Depth             : num  306 186 148 177 458 ...
##  $ SBT               : num  2.41 2.42 2.41 2.48 2.27 ...
##  $ SBS               : num  34.4 34.4 34.4 34.4 34.5 ...
##  $ Chl               : num  0.978 0.947 0.808 0.958 0.696 ...
##  $ SBT_var           : num  0.1197 0.0864 0.0927 0.1021 0.0905 ...
```

```
##  $ Chl_var           : num  1.35 1.38 1.17 1.42 1.29 ...
##  $ Fishing           : num  0.1294 0.0545 0.1055 0.1245 0.0567 ...
##  $ life_history_class: chr  "slow" "slow" "slow" "slow" ...
##  $ split             : chr  "train" "train" "train" "train" ...
##  $ spatial_block     : chr  "116_47" "116_47" "116_47" "116_47" ...
```

```
table(fish$life_history_class)
```

```
##
## fast slow
## 2006 2006
```

```
prop.table(
  table(fish$life_history_class)
)
```

```
##
## fast slow
##  0.5  0.5
```

Question 1

1. How many observations are in the dataset? **There are 4012 observations in the data set.**
2. Is the target approximately balanced? **The target is exactly balanced with 2006 observations of each.**

3. Explore environmental differences between classes

```
predictors <- c(
  "Depth",
  "SBT",
  "SBS",
  "Chl",
  "SBT_var",
  "Chl_var",
  "Fishing"
)

aggregate(
  fish[, predictors],
  by = list(
    life_history_class = fish$life_history_class
  ),
  FUN = mean,
  na.rm = TRUE
)
```

```
##   life_history_class    Depth       SBT      SBS       Chl  SBT_var  Chl_var   Fishing
## 1               fast 103.2957 12.323275 34.87949 0.8665125 5.931811 2.490550 0.5032422
## 2               slow 320.4782  4.084968 34.17179 0.9129641 1.657964 2.923874 0.4139380
```

```
variables_to_plot <- c(
  "SBT",
  "Depth",
  "SBS",
  "Chl"
```

```
)

par(
  mfrow = c(2, 2),
  mar = c(4, 4, 3, 1)
)

for (variable in variables_to_plot) {

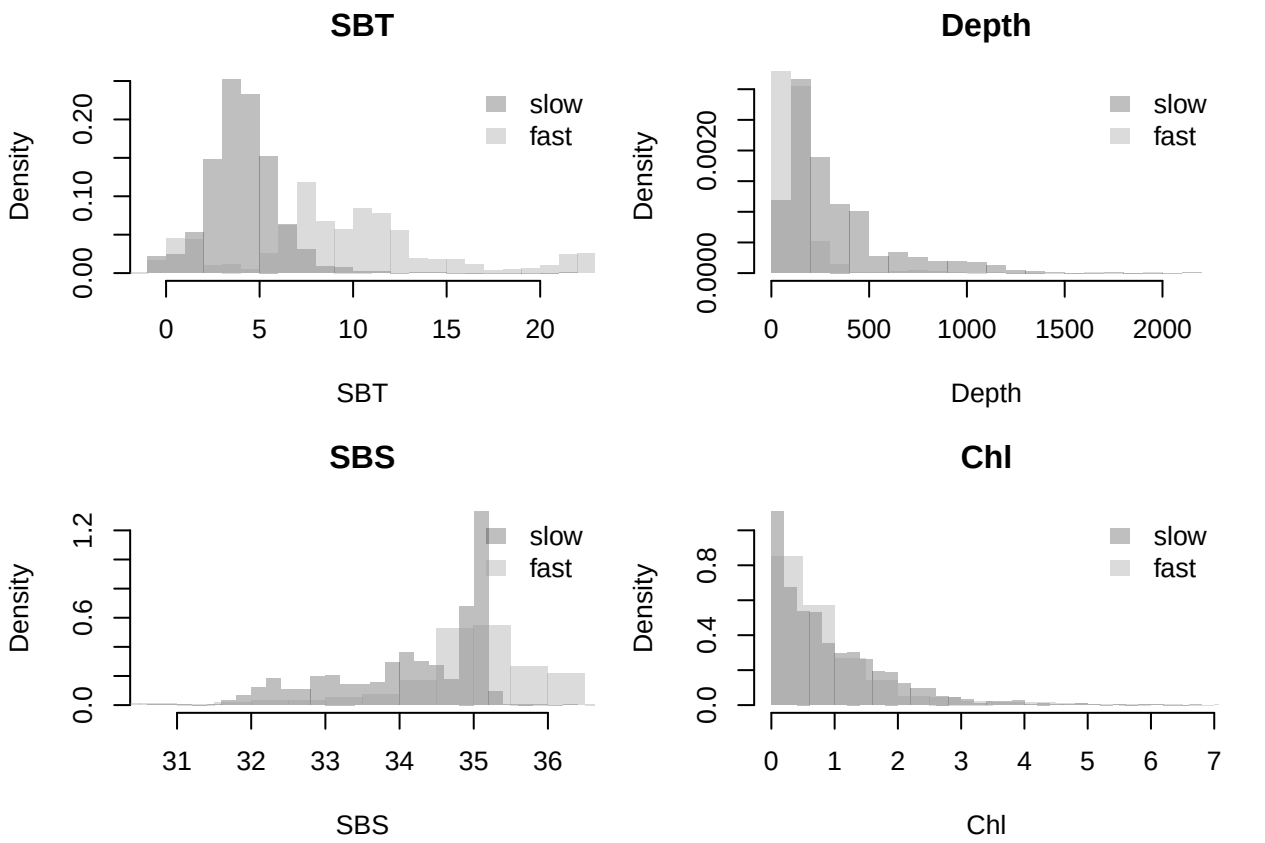
  slow_values <- fish[
    fish$life_history_class == "slow",
    variable
  ]

  fast_values <- fish[
    fish$life_history_class == "fast",
    variable
  ]

  hist(
    slow_values,
    breaks = 25,
    probability = TRUE,
    main = variable,
    xlab = variable,
    col = rgb(0, 0, 0, 0.25),
    border = NA
  )

  hist(
    fast_values,
    breaks = 25,
    probability = TRUE,
    add = TRUE,
    col = rgb(0.6, 0.6, 0.6, 0.35),
    border = NA
  )

  legend(
    "topright",
    legend = c("slow", "fast"),
    fill = c(
      rgb(0, 0, 0, 0.25),
      rgb(0.6, 0.6, 0.6, 0.35)
    ),
    border = NA,
    bty = "n"
  )
}
```




```
par(mfrow = c(1, 1))
```

Question 2

Describe at least **two environmental variables** that appear to differ between fast- and slow-life-history communities.

Do these patterns prove that the environmental variable *causes* the life-history strategy? Explain. SBT (sea bottom temperature) and Depth seem to differ between slow and fast life history fish. Depth and SBT are going to be correlated with lower temperatures occuring in deeper waters. These histograms alone cannot show causation. However, typically warmer water increases respiration and fish with faster life histories can survive and reproduce in warmer/shallow water because the sped up life history matches the increased respiration. Not a cause, but with knowledge of fish physiology it is an adaptation to suriviing in those environments.

4. Define predictors and target

```
X <- as.matrix(
  fish[, predictors]
)

y <- ifelse(
  fish$life_history_class == "fast",
  1,
  0
)
```

```r
dim(X)
```

```
## [1] 4012    7
```

```r
table(y)
```

```
## y
##    0    1
## 2006 2006
```

5. Use the instructor-provided training, validation, and test sets

Use:

- `train` for model fitting,
- `validation` for model selection and early stopping,
- `test` for final evaluation only.

The test set must remain untouched while you compare model settings.

```r
train_mask <- fish$split == "train"
validation_mask <- fish$split == "validation"
test_mask <- fish$split == "test"

X_fit <- X[
  train_mask,
  ,
  drop = FALSE
]

X_validation <- X[
  validation_mask,
  ,
  drop = FALSE
]

X_test <- X[
  test_mask,
  ,
  drop = FALSE
]

y_fit <- y[train_mask]
y_validation <- y[validation_mask]
y_test <- y[test_mask]

cat("Fit:", nrow(X_fit), "\n")
```

```
## Fit: 2801
```

```r
cat("Validation:", nrow(X_validation), "\n")
```

```
## Validation: 618
```

```r
cat("Test:", nrow(X_test), "\n")
```

```
## Test: 593
```

```r
table(
  fish$split,
  fish$life_history_class
)
```

```
##
##              fast slow
##   test        296  297
##   train      1398 1403
##   validation  312  306
```

6. Scale the environmental predictors

Fit the scaling parameters using the **training data only**.

```r
fit_means <- colMeans(X_fit)
fit_sds <- apply(
  X_fit,
  2,
  sd
)

X_fit_scaled <- sweep(
  sweep(
    X_fit,
    2,
    fit_means,
    "-"
  ),
  2,
  fit_sds,
  "/"
)

X_validation_scaled <- sweep(
  sweep(
    X_validation,
    2,
    fit_means,
    "-"
  ),
  2,
  fit_sds,
  "/"
)

X_test_scaled <- sweep(
  sweep(
    X_test,
    2,
    fit_means,
    "-"
  ),
  2,
```

```
  fit_sds,
  "/"
)
```

7. Establish a logistic-regression baseline

```
logistic_fit_data <- data.frame(
  y = y_fit,
  X_fit_scaled
)

logistic_model <- glm(
  y ~ .,
  data = logistic_fit_data,
  family = binomial()
)

logistic_validation_probability <- predict(
  logistic_model,
  newdata = data.frame(
    X_validation_scaled
  ),
  type = "response"
)

logistic_validation_pred <- ifelse(
  logistic_validation_probability >= 0.5,
  1,
  0
)
```

We will use the following metric functions throughout the homework.

```
accuracy_metric <- function(
  truth,
  prediction
) {
  mean(
    truth == prediction
  )
}

balanced_accuracy_metric <- function(
  truth,
  prediction
) {

  sensitivity <- mean(
    prediction[truth == 1] == 1
  )

  specificity <- mean(
    prediction[truth == 0] == 0
```

```
  )

  mean(
    c(
      sensitivity,
      specificity
    )
  )
}

roc_auc_metric <- function(
  truth,
  probability
) {

  n_positive <- sum(
    truth == 1
  )

  n_negative <- sum(
    truth == 0
  )

  probability_ranks <- rank(
    probability,
    ties.method = "average"
  )

  (
    sum(
      probability_ranks[
        truth == 1
      ]
    ) -
      n_positive *
        (n_positive + 1) /
        2
  ) /
    (
      n_positive *
        n_negative
    )
}
```

```
cat(
  "Validation accuracy:",
  accuracy_metric(
    y_validation,
    logistic_validation_pred
  ),
  "\n"
)
```

```
## Validation accuracy: 0.9255663
```

```
cat(
  "Validation balanced accuracy:",
  balanced_accuracy_metric(
    y_validation,
    logistic_validation_pred
  ),
  "\n"
)
```

```
## Validation balanced accuracy: 0.9257164
```

```
cat(
  "Validation ROC AUC:",
  roc_auc_metric(
    y_validation,
    logistic_validation_probability
  ),
  "\n"
)
```

```
## Validation ROC AUC: 0.9805807
```

Question 3

Why is logistic regression a useful baseline before fitting an MLP? **Starting with a logistic regression allows a less complex baseline model of linear categorization. Then we can determine if the hidden layers and complexity are useful in improving the model.**

What kind of decision relationship can an MLP represent that logistic regression cannot represent without manually creating nonlinear features?

MLPs can use non-linear relationships between multiple layers of data to predict outcomes.

8. Helper functions for the neural network

The R `torch` model will output **logits** rather than probabilities. During training, we will use `nn_bce_with_logits_loss()`, which combines the sigmoid transformation and binary cross-entropy loss in a numerically stable way.

```
predict_torch_probability <- function(
  model,
  X
) {

  model$eval()

  X_tensor <- torch_tensor(
    X,
    dtype = torch_float32()
  )

  probability <- with_no_grad({

    logits <- model(
      X_tensor
    )
```

```
    torch_sigmoid(
      logits
    )
  })

  as.numeric(
    as_array(
      probability
    )
  )
}
```

```
binary_cross_entropy <- function(
  truth,
  probability
) {

  epsilon <- 1e-7

  probability <- pmin(
    pmax(
      probability,
      epsilon
    ),
    1 - epsilon
  )

  -mean(
    truth *
      log(probability) +
      (1 - truth) *
      log(
        1 - probability
      )
  )
}
```

The training function below is supplied for you. It handles:

- Adam optimization,
- binary cross-entropy with logits,
- training and validation metrics,
- early stopping,
- `min_delta`,
- patience,
- and restoring the model from the epoch with the best validation loss.

```
train_torch_model <- function(
  model,
  X_fit,
  y_fit,
  X_validation = NULL,
  y_validation = NULL,
  epochs = 200,
  learning_rate = 0.001,
```

```
  weight_decay = 0,
  early_stopping = FALSE,
  patience = 20,
  min_delta = 1e-4,
  restore_best_weights = TRUE,
  verbose = FALSE
) {

  optimizer <- optim_adam(
    model$parameters,
    lr = learning_rate,
    weight_decay = weight_decay
  )

  loss_fn <- nn_bce_with_logits_loss()

  X_fit_tensor <- torch_tensor(
    X_fit,
    dtype = torch_float32()
  )

  y_fit_tensor <- torch_tensor(
    matrix(
      y_fit,
      ncol = 1
    ),
    dtype = torch_float32()
  )

  history <- data.frame(
    epoch = integer(),
    loss = numeric(),
    accuracy = numeric(),
    val_loss = numeric(),
    val_accuracy = numeric()
  )

  best_val_loss <- Inf
  best_epoch <- NA_integer_
  wait <- 0L

  best_model_path <- tempfile(
    fileext = ".pt"
  )

  for (epoch in seq_len(epochs)) {

    model$train()

    optimizer$zero_grad()

    logits <- model(
      X_fit_tensor
```

```
  )

  loss <- loss_fn(
    logits,
    y_fit_tensor
  )

  loss$backward()

  optimizer$step()

  train_probability <- predict_torch_probability(
    model,
    X_fit
  )

  train_prediction <- ifelse(
    train_probability >= 0.5,
    1,
    0
  )

  train_loss_value <- binary_cross_entropy(
    y_fit,
    train_probability
  )

  train_accuracy_value <- accuracy_metric(
    y_fit,
    train_prediction
  )

  if (!is.null(X_validation)) {

    validation_probability <- predict_torch_probability(
      model,
      X_validation
    )

    validation_prediction <- ifelse(
      validation_probability >= 0.5,
      1,
      0
    )

    validation_loss_value <- binary_cross_entropy(
      y_validation,
      validation_probability
    )

    validation_accuracy_value <- accuracy_metric(
      y_validation,
      validation_prediction
```

```
    )

  } else {

    validation_loss_value <- NA_real_
    validation_accuracy_value <- NA_real_
  }

  history <- rbind(
    history,
    data.frame(
      epoch = epoch,
      loss = train_loss_value,
      accuracy = train_accuracy_value,
      val_loss = validation_loss_value,
      val_accuracy = validation_accuracy_value
    )
  )

  if (
    early_stopping &&
    !is.null(X_validation)
  ) {

    if (
      validation_loss_value <
        best_val_loss -
        min_delta
    ) {

      best_val_loss <- validation_loss_value
      best_epoch <- epoch
      wait <- 0L

      torch_save(
        model,
        best_model_path
      )

    } else {

      wait <- wait + 1L
    }

    if (
      wait >= patience
    ) {

      if (verbose) {
        cat(
          "Early stopping at epoch",
          epoch,
          "\n"
```

```
        )
      }

      break
    }
  }

  if (
    verbose &&
    epoch %% 50 == 0
  ) {

    cat(
      "Epoch:",
      epoch,
      "validation loss:",
      round(
        validation_loss_value,
        4
      ),
      "\n"
    )
  }
}

stopped_epoch <- max(
  history$epoch
)

if (
  early_stopping &&
  restore_best_weights &&
  file.exists(
    best_model_path
  )
) {

  model <- torch_load(
    best_model_path
  )
}

if (
  file.exists(
    best_model_path
  )
) {

  unlink(
    best_model_path
  )
}
```

```
  list(
    model = model,
    history = history,
    best_epoch = best_epoch,
    stopped_epoch = stopped_epoch
  )
}
```

9. Build your first MLP

Build a network with:

- 7 environmental inputs,
- one hidden layer with **8 units**,
- ReLU activation in the hidden layer,
- one output unit,
- Adam optimization,
- binary cross-entropy loss.

Remember: the output layer should return a **logit**. The sigmoid transformation is handled by the loss function during training and by `predict_torch_probability()` during prediction.

Complete the missing lines.

```
mlp_net <- nn_module(
  "MLPNet",

  initialize = function(
    hidden_units = 8,
    dropout_rate = 0
  ) {

    # TODO: create the hidden layer
    self$hidden <- nn_linear(7, hidden_units)

    self$dropout <- nn_dropout(
      p = dropout_rate
    )

    # TODO: create the one-unit output layer
    self$output <- nn_linear(hidden_units, 1)
  },

  forward = function(x) {

    # TODO: apply the hidden layer
    x <- self$hidden(x)

    # TODO: apply ReLU
    x <- nnf_relu(x)

    x <- self$dropout(
      x
    )
```

```
    self$output(
      x
    )
  }
)
```

```
set.seed(42)
torch_manual_seed(42)

model <- mlp_net(
  hidden_units = 8,
  dropout_rate = 0
)

model
```

```
## An `nn_module` containing 73 parameters.
##
## -- Modules -----------------------------------------------------------------------------------------
## * hidden: <nn_linear> #64 parameters
## * dropout: <nn_dropout> #0 parameters
## * output: <nn_linear> #9 parameters
```

10. Train the MLP

```
fitted_model <- train_torch_model(
  model = model,
  X_fit = X_fit_scaled,
  y_fit = y_fit,
  X_validation = X_validation_scaled,
  y_validation = y_validation,
  epochs = 200,
  learning_rate = 0.01
)

history_df <- fitted_model$history
model <- fitted_model$model
```

11. Visualize the learning history

```
par(
  mfrow = c(1, 2)
)

plot(
  history_df$epoch,
  history_df$loss,
  type = "l",
  xlab = "Epoch",
  ylab = "Binary cross-entropy loss",
  main = "Loss"
)
```

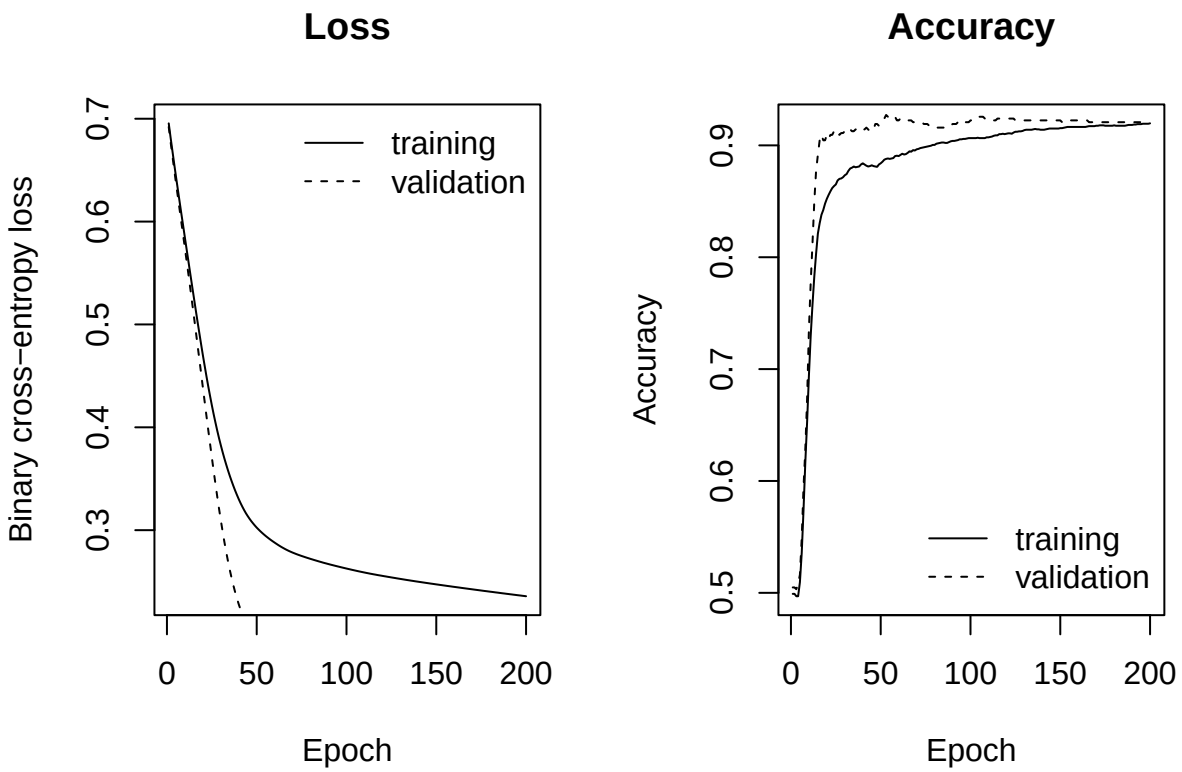
```
lines(
  history_df$epoch,
  history_df$val_loss,
  lty = 2
)

legend(
  "topright",
  legend = c(
    "training",
    "validation"
  ),
  lty = c(
    1,
    2
  ),
  bty = "n"
)

plot(
  history_df$epoch,
  history_df$accuracy,
  type = "l",
  xlab = "Epoch",
  ylab = "Accuracy",
  main = "Accuracy"
)

lines(
  history_df$epoch,
  history_df$val_accuracy,
  lty = 2
)

legend(
  "bottomright",
  legend = c(
    "training",
    "validation"
  ),
  lty = c(
    1,
    2
  ),
  bty = "n"
)
```




```
par(
  mfrow = c(1, 1)
)
```

Question 4

Examine both plots.

1. Does the model appear to be underfitting, overfitting, or reasonably fit? **It seems reasonably fit, the logisitic regression had high accuracy, so I was expecting extremely high accuracy from the MLP.**
2. Around what epoch does validation loss stop improving substantially? **50**
3. Why can validation accuracy remain nearly constant while validation loss continues to change? **I think this means while training data continues to change. The model is informing changes in the next epoch in the training model through the loss, which results in more accurate predicitions each epoch. The validation data is not informing the model, so the loss is not changing, because the model has already been built.**

12. Add early stopping

Now train the same model with early stopping.

Monitor **validation loss** and use:

- `patience = 20`
- `min_delta = 1e-4`
- `restore_best_weights = TRUE`
- maximum of 1000 epochs

Fill in the missing values.

```r
set.seed(42)
torch_manual_seed(42)

early_model <- mlp_net(
  hidden_units = 8,
  dropout_rate = 0
)

early_fitted <- train_torch_model(
  model = early_model,
  X_fit = X_fit_scaled,
  y_fit = y_fit,
  X_validation = X_validation_scaled,
  y_validation = y_validation,
  epochs = 3333,                # TODO
  learning_rate = 0.01,
  early_stopping = TRUE,
   patience = 20, # TODO
  min_delta = 1e-4, # TODO
  restore_best_weights = TRUE,  # TODO
)

early_history_df <- early_fitted$history
early_model <- early_fitted$model
```

```r
best_epoch <- early_fitted$best_epoch
stopped_epoch <- early_fitted$stopped_epoch

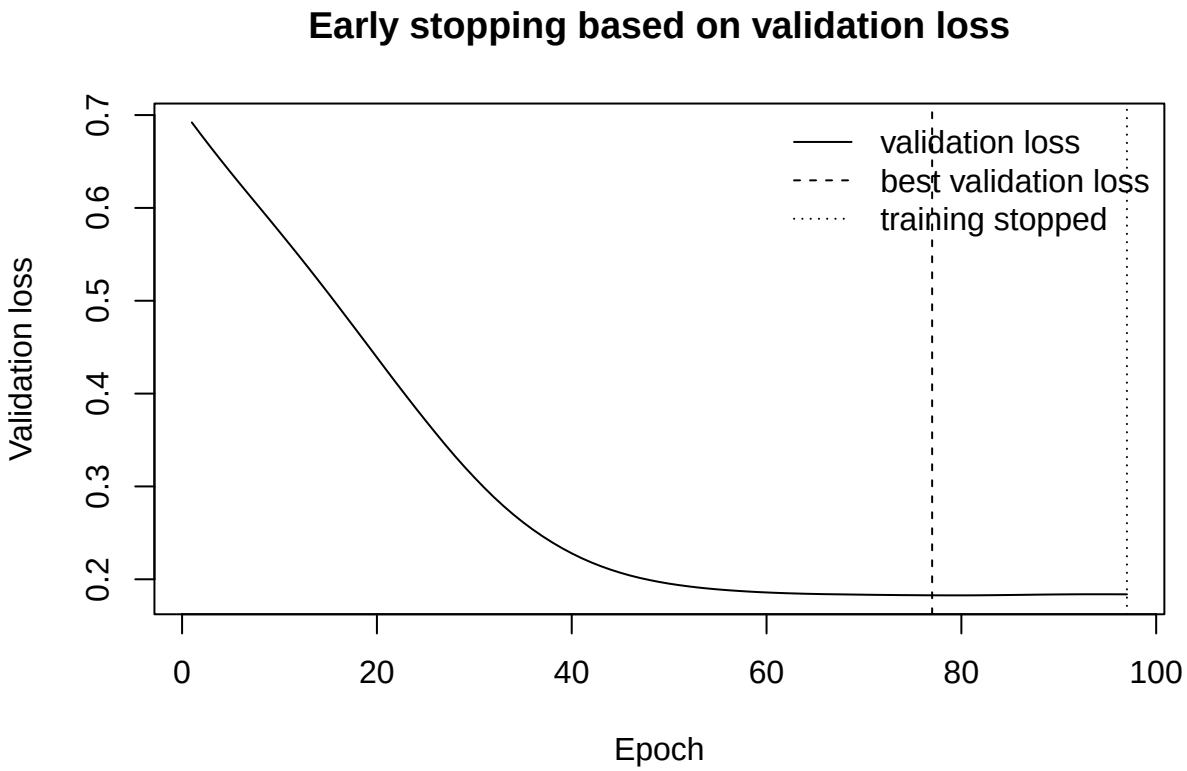
plot(
  early_history_df$epoch,
  early_history_df$val_loss,
  type = "l",
  xlab = "Epoch",
  ylab = "Validation loss",
  main = "Early stopping based on validation loss"
)

abline(
  v = best_epoch,
  lty = 2
)

abline(
  v = stopped_epoch,
  lty = 3
)

legend(
  "topright",
  legend = c(
    "validation loss",
    "best validation loss",
    "training stopped"
  ),
```

```
  lty = c(
    1,
    2,
    3
  ),
  bty = "n"
)
```




```
cat(
  "Best epoch:",
  best_epoch,
  "\n"
)
```

```
## Best epoch: 77
```

```
cat(
  "Stopped epoch:",
  stopped_epoch,
  "\n"
)
```

```
## Stopped epoch: 97
```

Question 5

1. At what epoch was validation loss lowest? **77**
2. At what epoch did training stop? **97**
3. Why are those two epochs not necessarily identical? **Training continued for 20 epochs to make sure there was not a lower validation loss achieved**

4. What does `restore_best_weights = TRUE` do? **It replaces the 97 epochs weights with the acheived lowest weights from epoch 77**

13. Experiment with model complexity

The helper below builds the same MLP with different hidden-unit, dropout, and L2 settings.

In R `torch`, L2 regularization is supplied to Adam as `weight_decay`.

```
build_mlp <- function(
  hidden_units = 8,
  dropout_rate = 0
) {

  mlp_net(
    hidden_units = hidden_units,
    dropout_rate = dropout_rate
  )
}
```

```
evaluate_setting <- function(
  label,
  hidden_units = 8,
  dropout_rate = 0,
  l2_strength = 0
) {

  set.seed(42)
  torch_manual_seed(42)

  model <- build_mlp(
    hidden_units = hidden_units,
    dropout_rate = dropout_rate
  )

  fitted <- train_torch_model(
    model = model,
    X_fit = X_fit_scaled,
    y_fit = y_fit,
    X_validation = X_validation_scaled,
    y_validation = y_validation,
    epochs = 1000,
    learning_rate = 0.01,
    weight_decay = l2_strength,
    early_stopping = TRUE,
    patience = 20,
    min_delta = 1e-4,
    restore_best_weights = TRUE
  )

  probability <- predict_torch_probability(
    fitted$model,
    X_validation_scaled
  )
```

```
  prediction <- ifelse(
    probability >= 0.5,
    1,
    0
  )

  data.frame(
    setting = label,
    hidden_units = hidden_units,
    dropout = dropout_rate,
    l2 = l2_strength,
    epochs_used = fitted$stopped_epoch,
    validation_accuracy = accuracy_metric(
      y_validation,
      prediction
    ),
    validation_balanced_accuracy = balanced_accuracy_metric(
      y_validation,
      prediction
    ),
    validation_auc = roc_auc_metric(
      y_validation,
      probability
    ),
    best_validation_loss = min(
      fitted$history$val_loss
    )
  )
}
```

Change **one setting at a time**. For each experiment, train using the same fit and validation data and record the validation results.

Complete at least these experiments:

1. `hidden_units = 1`
2. `hidden_units = 32`
3. `hidden_units = 64`
4. baseline 8-unit model + dropout of `0.30`
5. baseline 8-unit model + L2 regularization

Use early stopping for each experiment.

```
experiments <- rbind(
    # TODO: add the required experimental settings here.
  #
  # Example:
  #
  # evaluate_setting(
  #   "1 hidden unit",
  #   hidden_units = 1
  # )
  # variables to change hidden_units, dropout_rate, and l2_strength
evaluate_setting(
  label="1. 1 hidden unit",
  hidden_units = 1,
```

```
  dropout_rate = 0,
  l2_strength = 0
),
evaluate_setting(
  label="2. 32 hidden units",
  hidden_units = 32,
  dropout_rate = 0,
  l2_strength = 0
),
evaluate_setting(
  label="3. 64 hidden units",
  hidden_units = 64,
  dropout_rate = 0,
  l2_strength = 0
),
evaluate_setting(
  label="4. 8 hidden units, dropout 0.30",
  hidden_units = 8,
  dropout_rate =  0.30,
  l2_strength = 0
),
evaluate_setting(
  label="5. 8 hidden units, L2 reg",
  hidden_units = 8,
  dropout_rate =  0,
  l2_strength = 0.001
)
)

results <- experiments

results
```

```
##                           setting hidden_units dropout    l2 epochs_used validation_accuracy
## 1                1. 1 hidden unit            1     0.0 0.000         626           0.9288026
## 2              2. 32 hidden units           32     0.0 0.000          74           0.9255663
## 3              3. 64 hidden units           64     0.0 0.000         183           0.9433657
## 4 4. 8 hidden units, dropout 0.30            8     0.3 0.000         329           0.9514563
## 5       5. 8 hidden units, L2 reg            8     0.0 0.001          97           0.9190939
##   validation_balanced_accuracy validation_auc best_validation_loss
## 1                    0.9290787      0.9806016            0.1945369
## 2                    0.9255279      0.9821099            0.1805209
## 3                    0.9434389      0.9875775            0.1413876
## 4                    0.9516403      0.9857340            0.1461555
## 5                    0.9191805      0.9811254            0.1836222
```

Question 6

Using only the **validation results**:

1. Does the 1-unit model appear to underfit? **No, it has the 3rd highest accuracy (92.9%).**
2. Does increasing the number of hidden units consistently improve validation performance? **No, 32 units has lower accuracy than 1.**
3. Did dropout help? **8 units with dropout had the highest accuracy (95.1%)**
4. Did L2 regularization help? **L2 regularization had the lowest accuracy.**

5. Which model setting would you carry forward for final evaluation, and why? **I would take the 8 hidden unit with dropout forward for final validation, because I think it would be the most reboust to avoid overfitting, with fewer layers and having nodes removed it should be able to do well with test data.**

14. Final model evaluation

Choose the model setting you selected using validation performance.

Refit that model using the same fit/validation workflow and early stopping. Then evaluate it **once** on the test set.

```
# TODO: replace these values with the settings you selected.

FINAL_HIDDEN_UNITS <- 8
FINAL_DROPOUT <- 0.3
FINAL_L2 <- 0
```

```
set.seed(42)
torch_manual_seed(42)

final_model <- build_mlp(
  hidden_units = FINAL_HIDDEN_UNITS,
  dropout_rate = FINAL_DROPOUT
)

final_fitted <- train_torch_model(
  model = final_model,
  X_fit = X_fit_scaled,
  y_fit = y_fit,
  X_validation = X_validation_scaled,
  y_validation = y_validation,
  epochs = 1000,
  learning_rate = 0.01,
  weight_decay = FINAL_L2,
  early_stopping = TRUE,
  patience = 20,
  min_delta = 1e-4,
  restore_best_weights = TRUE
)

final_model <- final_fitted$model

test_probability <- predict_torch_probability(
  final_model,
  X_test_scaled
)

test_pred <- ifelse(
  test_probability >= 0.5,
  1,
  0
)

cat(
```

```
  "Test accuracy:",
  accuracy_metric(
    y_test,
    test_pred
  ),
  "\n"
)
```

```
## Test accuracy: 0.9342327
```

```
cat(
  "Test balanced accuracy:",
  balanced_accuracy_metric(
    y_test,
    test_pred
  ),
  "\n"
)
```

```
## Test balanced accuracy: 0.9342012
```

```
cat(
  "Test ROC AUC:",
  roc_auc_metric(
    y_test,
    test_probability
  ),
  "\n"
)
```

```
## Test ROC AUC: 0.9766244
```

```
confusion_matrix <- table(
  Predicted = factor(
    test_pred,
    levels = c(0, 1),
    labels = c(
      "slow",
      "fast"
    )
  ),
  Observed = factor(
    y_test,
    levels = c(0, 1),
    labels = c(
      "slow",
      "fast"
    )
  )
)

confusion_matrix
```

```
##           Observed
## Predicted slow fast
##      slow  283   25
```

```
##      fast   14  271
```

15. Compare the MLP with logistic regression on the test set

Only now should you evaluate the logistic-regression baseline on the test set.

```
logistic_test_probability <- predict(
  logistic_model,
  newdata = data.frame(
    X_test_scaled
  ),
  type = "response"
)

logistic_test_pred <- ifelse(
  logistic_test_probability >= 0.5,
  1,
  0
)

comparison <- data.frame(
  model = c(
    "Logistic regression",
    "MLP"
  ),

  test_accuracy = c(
    accuracy_metric(
      y_test,
      logistic_test_pred
    ),
    accuracy_metric(
      y_test,
      test_pred
    )
  ),

  test_balanced_accuracy = c(
    balanced_accuracy_metric(
      y_test,
      logistic_test_pred
    ),
    balanced_accuracy_metric(
      y_test,
      test_pred
    )
  ),

  test_roc_auc = c(
    roc_auc_metric(
      y_test,
      logistic_test_probability
    ),
    roc_auc_metric(
```

```
      y_test,
      test_probability
    )
  )
)

comparison
```

```
##                 model test_accuracy test_balanced_accuracy test_roc_auc
## 1 Logistic regression     0.9173693              0.9173207    0.9542725
## 2                 MLP     0.9342327              0.9342012    0.9766244
```

Question 7

Did the MLP clearly outperform logistic regression?

Your answer should consider:

- accuracy,
- balanced accuracy,
- ROC AUC,
- model complexity,
- and whether the improvement is large enough to justify the more complicated model.

A neural network is not automatically the better scientific model simply because it is more flexible. **The MLP did not clearly outperform the logisitic regression. The accuracy improved by about ~2%. The logisitic regression had such a high accuracy, I don't think I would have pursued a MLP on my own. There are many decisions that go into the MLP that could lead to an overfit model. I think a simpler model with the ability to compare future observations and predictability would be more useful than the hidden layers and increased complexity of an MLP. For example, I'm sure an MLP may be able to capture climate change impacts, however, knowing the differences in environmental variables and their sized impact on the model would not be possible.**

Grading guide

Component	Points
Data inspection	1
Train/validation/test design and scaling	1
Logistic-regression baseline	1
Correct MLP construction and training	2
Learning curves and early stopping	4
Model-complexity / regularization experiments	4
Final test evaluation and baseline comparison	2
Questions 1-7	10
Total	**25**